

Our first command object

Isn't it about time we built our first command object? Let's go ahead and write some code for the remote control. While we haven't figured out how to design the remote control API yet, building a few things from the bottom up may help us...



Implementing the Command interface

First things first: all command objects implement the same interface, which consists of one method. In the Diner we called this method `orderUp()`; however, we typically just use the name `execute()`.

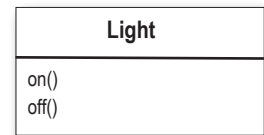
Here's the Command interface:

```
public interface Command {
    public void execute();
}
```

Simple. All we need is one method called `execute()`.

Implementing a command to turn a light on

Now, let's say you want to implement a command for turning a light on. Referring to our set of vendor classes, the `Light` class has two methods: `on()` and `off()`. Here's how you can implement this as a command:



```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }
}
```

This is a command, so we need to implement the Command interface.

The constructor is passed the specific light that this command is going to control—say the living room light—and stashes it in the `light` instance variable. When `execute` gets called, this is the light object that is going to be the receiver of the request.

The `execute()` method calls the `on()` method on the receiving object, which is the light we are controlling.

Now that you've got a `LightOnCommand` class, let's see if we can put it to use...

Using the command object

Okay, let's make things simple: say we've got a remote control with only one button and corresponding slot to hold a device to control:

```
public class SimpleRemoteControl {
    Command slot;
    public SimpleRemoteControl() {}

    public void setCommand(Command command) {
        slot = command;
    }

    public void buttonWasPressed() {
        slot.execute();
    }
}
```

We have one slot to hold our command, which will control one device.

We have a method for setting the command the slot is going to control. This could be called multiple times if the client of this code wanted to change the behavior of the remote button.

This method is called when the button is pressed. All we do is take the current command bound to the slot and call its execute() method.

Creating a simple test to use the Remote Control

Here's just a bit of code to test out the simple remote control. Let's take a look and we'll point out how the pieces match the Command Pattern diagram:

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        LightOnCommand lightOn = new LightOnCommand(light);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
    }
}
```

This is our Client in Command Pattern-speak.

The remote is our Invoker; it will be passed a command object that can be used to make requests.

Now we create a Light object. This will be the Receiver of the request.

Here, create a command and pass the Receiver to it.

Here, pass the command to the Invoker.

And then we simulate the button being pressed.

Here's the output of running this test code.

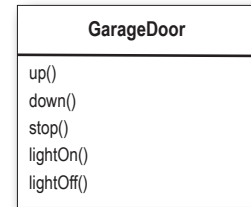
```
File Edit Window Help DinerFoodYum
%java RemoteControlTest

Light is On
%
```



Okay, it's time for you to implement the `GarageDoorOpenCommand` class. First, supply the code for the class below. You'll need the `GarageDoor` class diagram.

```
public class GarageDoorOpenCommand
    implements Command {
```



← Your code here

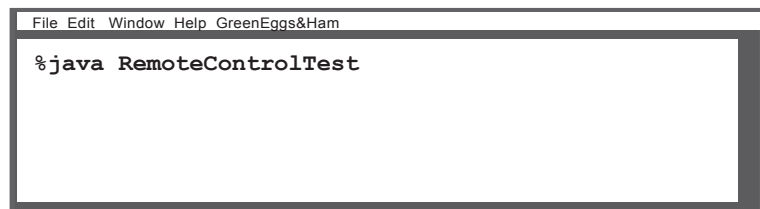
```
}
```

Now that you've got your class, what is the output of the following code? (Hint: the `GarageDoor up()` method prints out "Garage Door is Open" when it is complete.)

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        GarageDoor garageDoor = new GarageDoor();
        LightOnCommand lightOn = new LightOnCommand(light);
        GarageDoorOpenCommand garageOpen =
            new GarageDoorOpenCommand(garageDoor);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
        remote.setCommand(garageOpen);
        remote.buttonWasPressed();
    }
}
```

Your output here. →



The Command Pattern defined

You've done your time in the Objectville Diner, you've partly implemented the remote control API, and in the process you've got a fairly good picture of how the classes and objects interact in the Command Pattern. Now we're going to define the Command Pattern and nail down all the details.

Let's start with its official definition:

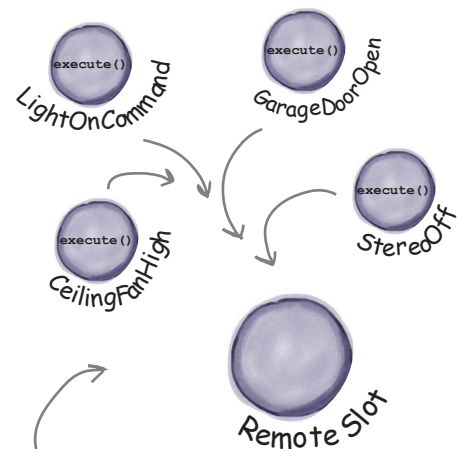
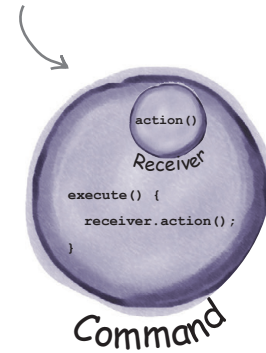
The Command Pattern encapsulates a request as an object, thereby letting you parameterize other objects with different requests, queue or log requests, and support undoable operations.

Let's step through this. We know that a command object *encapsulates a request* by binding together a set of actions on a specific receiver. To achieve this, it packages the actions and the receiver into an object that exposes just one method, `execute()`. When called, `execute()` causes the actions to be invoked on the receiver. From the outside, no other objects really know what actions get performed on what receiver; they just know that if they call the `execute()` method, their request will be serviced.

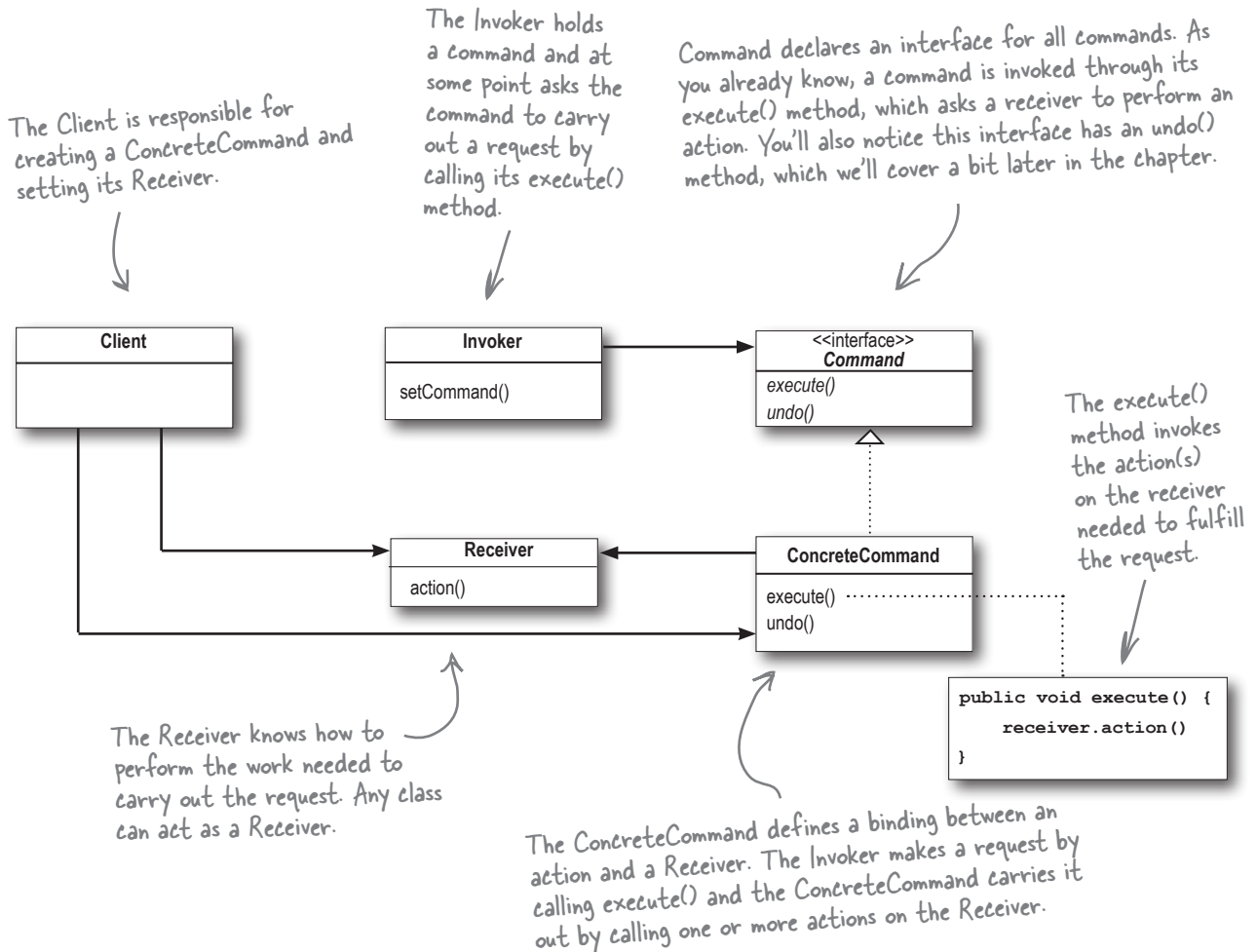
We've also seen a couple examples of *parameterizing an object* with a command. Back at the diner, the Waitress was parameterized with multiple orders throughout the day. In the simple remote control, we first loaded the button slot with a "light on" command and then later replaced it with a "garage door open" command. Like the Waitress, your remote slot didn't care what command object it had, as long as it implemented the Command interface.

What we haven't encountered yet is using commands to implement *queues and logs and support undo operations*. Don't worry, those are pretty straightforward extensions of the basic Command Pattern, and we'll get to them soon. We can also easily support what's known as the Meta Command Pattern once we have the basics in place. The Meta Command Pattern allows you to create macros of commands so that you can execute multiple commands at once.

An encapsulated request.



The Command Pattern defined: the class diagram



How does the design of the Command Pattern support the decoupling of the invoker of a request and the receiver of the request?

Okay, I think I've got a good feel for the Command Pattern now. Great tip, Joe, I think we're going to look like superstars after finishing off the Remote Control API.

Mary: Me too. So where do we begin?

Sue: Like we did in the SimpleRemote, we need to provide a way to assign commands to slots. In our case we have seven slots, each with an on and off button. So we might assign commands to the remote something like this:

```
onCommands[0] = onCommand;  
offCommands[0] = offCommand;
```

and so on for each of the seven command slots.

Mary: That makes sense, except for the Light objects. How does the remote know the living room from the kitchen light?

Sue: Ah, that's just it—it doesn't! The remote doesn't know anything but how to call `execute()` on the corresponding command object when a button is pressed.

Mary: Yeah, I sorta got that, but in the implementation, how do we make sure the right objects are turning on and off the right devices?

Sue: When we create the commands to be loaded into the remote, we create one `LightCommand` that is bound to the living room light object and another that is bound to the kitchen light object. Remember, the receiver of the request gets bound to the command it's encapsulated in. So, by the time the button is pressed, no one cares which light is which; the right thing just happens when the `execute()` method is called.

Mary: I think I've got it. Let's implement the remote and I think this will get clearer!

Sue: Sounds good. Let's give it a shot...

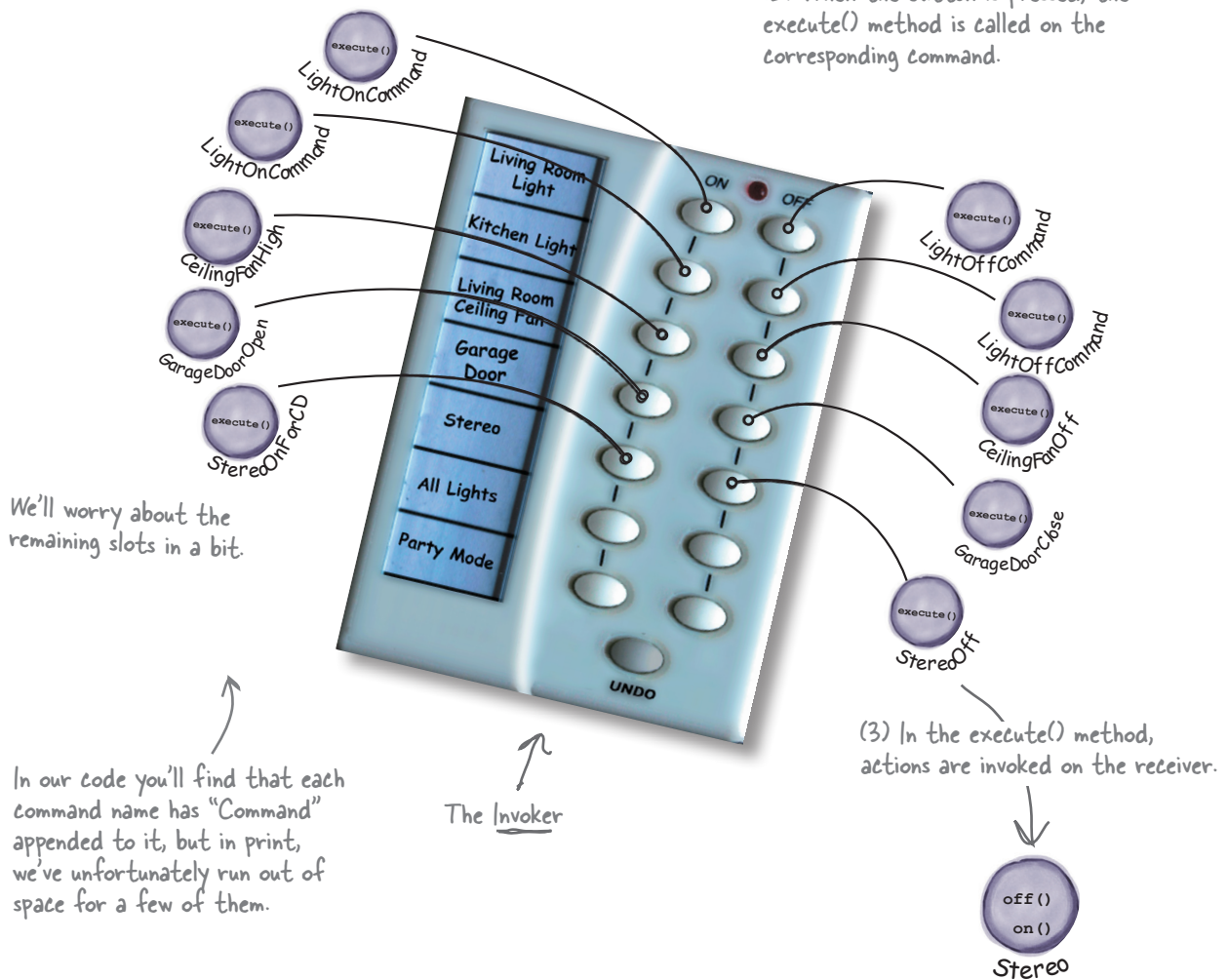


Assigning Commands to slots

So we have a plan: we're going to assign a command to each slot in the remote control. This makes the remote control our *invoker*. When a button is pressed, the `execute()` method will be called on the corresponding command, which results in actions being invoked on the receiver (like lights, ceiling fans, and stereos).

(1) Each slot gets a command.

(2) When the button is pressed, the `execute()` method is called on the corresponding command.



Implementing the Remote Control

```
public class RemoteControl {  
    Command[] onCommands;  
    Command[] offCommands;
```

← This time around, the remote is going to handle seven On and Off commands, which we'll hold in corresponding arrays.

```
    public RemoteControl() {  
        onCommands = new Command[7];  
        offCommands = new Command[7];
```

↙ ↘ In the constructor, all we need to do is instantiate and initialize the On and Off arrays.

```
        Command noCommand = new NoCommand();  
        for (int i = 0; i < 7; i++) {  
            onCommands[i] = noCommand;  
            offCommands[i] = noCommand;  
        }  
    }
```

↙ ↘ The setCommand() method takes a slot position and an On and Off command to be stored in that slot.

```
    public void setCommand(int slot, Command onCommand, Command offCommand) {  
        onCommands[slot] = onCommand;  
        offCommands[slot] = offCommand;  
    }
```

← It puts these commands in the On and Off arrays for later use.

```
    public void onButtonWasPushed(int slot) {  
        onCommands[slot].execute();  
    }
```

```
    public void offButtonWasPushed(int slot) {  
        offCommands[slot].execute();  
    }
```

↙ ↘ When an On or Off button is pressed, the hardware takes care of calling the corresponding methods onButtonWasPushed() or offButtonWasPushed().

```
    public String toString() {  
        StringBuffer stringBuff = new StringBuffer();  
        stringBuff.append("\n----- Remote Control -----\n");  
        for (int i = 0; i < onCommands.length; i++) {  
            stringBuff.append("[slot " + i + "] " + onCommands[i].getClass().getName()  
                + " " + offCommands[i].getClass().getName() + "\n");  
        }  
        return stringBuff.toString();  
    }  
}
```

↙ We override toString() to print out each slot and its corresponding command. You'll see us use this when we test the remote control.

Implementing the Commands

Well, we've already gotten our feet wet implementing the `LightOnCommand` for the `SimpleRemoteControl`. We can plug that same code in here and everything works beautifully. Off commands are no different; in fact, the `LightOffCommand` looks like this:

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }
}
```

The `LightOffCommand` works exactly the same way as the `LightOnCommand`, except that we're binding the receiver to a different action: the `off()` method.

Let's try something a little more challenging; how about writing on and off commands for the `Stereo`? Okay, off is easy, we just bind the `Stereo` to the `off()` method in the `StereoOffCommand`. On is a little more complicated; let's say we want to write a `StereoOnWithCDCommand`...

```
public class StereoOnWithCDCommand implements Command {
    Stereo stereo;

    public StereoOnWithCDCommand(Stereo stereo) {
        this.stereo = stereo;
    }

    public void execute() {
        stereo.on();
        stereo.setCD();
        stereo.setVolume(11);
    }
}
```

Just like the `LightOnCommand`, we get passed the instance of the `stereo` we're going to be controlling and we store it in an instance variable.

To carry out this request, we need to call three methods on the `stereo`: first, turn it on, then set it to play the CD, and finally set the volume to 11. Why 11? Well, it's better than 10, right?

Stereo
<code>on()</code> <code>off()</code> <code>setCd()</code> <code>setDvd()</code> <code>setRadio()</code> <code>setVolume()</code>

Not too bad. Take a look at the rest of the vendor classes; by now, you can definitely knock out the rest of the Command classes we need for those.

Putting the Remote Control through its paces

Our job with the remote is pretty much done; all we need to do is run some tests and get some documentation together to describe the API. Home Automation or Bust, Inc., sure is going to be impressed, don't ya think? We've managed to come up with a design that will allow them to produce a remote that is easy to maintain, and they're going to have no trouble convincing the vendors to write some simple command classes in the future since those are so easy to write.

Let's get to testing this code!

```
public class RemoteLoader {
```

```
    public static void main(String[] args) {
        RemoteControl remoteControl = new RemoteControl();
```

```
        Light livingRoomLight = new Light("Living Room");
        Light kitchenLight = new Light("Kitchen");
        CeilingFan ceilingFan = new CeilingFan("Living Room");
        GarageDoor garageDoor = new GarageDoor("Garage");
        Stereo stereo = new Stereo("Living Room");
```

Create all the devices in their proper locations.

```
        LightOnCommand livingRoomLightOn =
            new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
            new LightOffCommand(livingRoomLight);
        LightOnCommand kitchenLightOn =
            new LightOnCommand(kitchenLight);
        LightOffCommand kitchenLightOff =
            new LightOffCommand(kitchenLight);
```

Create all the Light Command objects.

```
        CeilingFanOnCommand ceilingFanOn =
            new CeilingFanOnCommand(ceilingFan);
        CeilingFanOffCommand ceilingFanOff =
            new CeilingFanOffCommand(ceilingFan);
```

Create the On and Off for the ceiling fan.

```
        GarageDoorUpCommand garageDoorUp =
            new GarageDoorUpCommand(garageDoor);
        GarageDoorDownCommand garageDoorDown =
            new GarageDoorDownCommand(garageDoor);
```

Create the Up and Down commands for the Garage.

```
        StereoOnWithCDCommand stereoOnWithCD =
            new StereoOnWithCDCommand(stereo);
        StereoOffCommand stereoOff =
            new StereoOffCommand(stereo);
```

Create the stereo On and Off commands.

```

remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);
remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);
remoteControl.setCommand(2, ceilingFanOn, ceilingFanOff);
remoteControl.setCommand(3, stereoOnWithCD, stereoOff);

System.out.println(remoteControl);

```

Now that we've got all our commands, we can load them into the remote slots.

```

remoteControl.onButtonWasPushed(0);
remoteControl.offButtonWasPushed(0);
remoteControl.onButtonWasPushed(1);
remoteControl.offButtonWasPushed(1);
remoteControl.onButtonWasPushed(2);
remoteControl.offButtonWasPushed(2);
remoteControl.onButtonWasPushed(3);
remoteControl.offButtonWasPushed(3);
}
}

```

Here's where we use our `toString()` method to print each remote slot and the command assigned to it. (Note that `toString()` gets called automatically here, so we don't have to call `toString()` explicitly.)

All right, we are ready to roll! Now, we step through each slot and push its On and Off buttons.

Now, let's check out the execution of our remote control test...

File Edit Window Help CommandsGetThingsDone

```

% java RemoteLoader
----- Remote Control -----
[slot 0] LightOnCommand          LightOffCommand
[slot 1] LightOnCommand          LightOffCommand
[slot 2] CeilingFanOnCommand     CeilingFanOffCommand
[slot 3] StereoOnWithCDCommand   StereoOffCommand
[slot 4] NoCommand               NoCommand
[slot 5] NoCommand               NoCommand
[slot 6] NoCommand               NoCommand

```

On slots Off slots

```

Living Room light is on
Living Room light is off
Kitchen light is on
Kitchen light is off
Living Room ceiling fan is on high
Living Room ceiling fan is off
Living Room stereo is on
Living Room stereo is set for CD input
Living Room stereo volume set to 11
Living Room stereo is off
%

```

Our commands in action! Remember, the output from each device comes from the vendor classes. For instance, when a light object is turned on, it prints "Living Room light is on."

Wait a second, what's with that NoCommand that's loaded in slots 4 through 6? Trying to pull a fast one?



Good catch. We did sneak a little something in there. In the remote control, we didn't want to check to see if a command was loaded every time we referenced a slot. For instance, in the `onButtonWasPushed()` method, we would need code like this:

```
public void onButtonWasPushed(int slot) {
    if (onCommands[slot] != null) {
        onCommands[slot].execute();
    }
}
```

So, how do we get around that? Implement a command that does nothing!

```
public class NoCommand implements Command {
    public void execute() { }
}
```

Then, in our `RemoteControl` constructor, we assign every slot a `NoCommand` object by default and we know we'll always have some command to call in each slot.

```
Command noCommand = new NoCommand();
for (int i = 0; i < 7; i++) {
    onCommands[i] = noCommand;
    offCommands[i] = noCommand;
}
```

So, in the output of our test run, you're seeing only slots that have been assigned to a command other than the default `NoCommand` object, which we assigned when we created the `RemoteControl` constructor.



Pattern Honorable Mention

The `NoCommand` object is an example of a *null object*. A null object is useful when you don't have a meaningful object to return, and yet you want to remove the responsibility for handling **null** from the client. For instance, in our remote control we didn't have a meaningful object to assign to each slot out of the box, so we provided a `NoCommand` object that acts as a surrogate and does nothing when its `execute()` method is called.

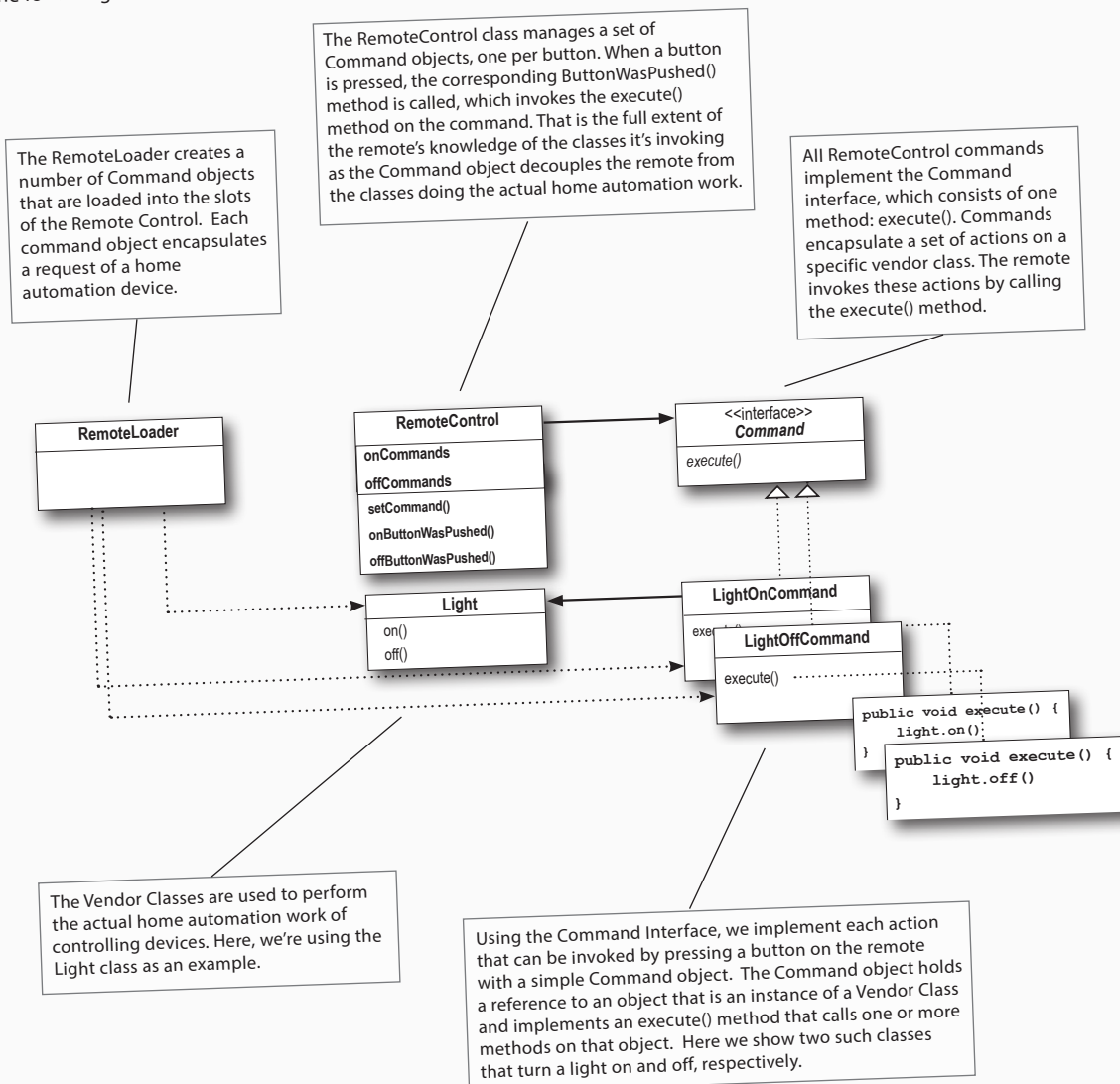
You'll find uses for Null Objects in conjunction with many Design Patterns, and sometimes you'll even see "Null Object" listed as a Design Pattern.

Time to write that documentation...

Remote Control API Design for Home Automation or Bust, Inc.

We are pleased to present you with the following design and application programming interface for your Home Automation Remote Control. Our primary design goal was to keep the remote control code as simple as possible so that it doesn't require changes as new vendor classes are produced. To this end we have employed the Command Pattern to logically decouple the RemoteControl class from the Vendor Classes. We believe this will reduce the cost of producing the remote as well as drastically reduce your ongoing maintenance costs.

The following class diagram provides an overview of our design:





Serious Coding

Want to take your Command Pattern coding to the next level? You can use Java's lambda expressions to skip the step of creating all those concrete command objects. With lambda expressions, instead of instantiating the concrete command objects, you can use *function objects* in their place. In other words, we can use a function object *as a command*. And, while we're at it, we can delete all those concrete Command classes, too.

Let's take a look at how you'd use lambda expressions as commands to simplify our previous code:

The updated code, using lambda expressions:

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControl remoteControl = new RemoteControl();

        Light livingRoomLight = new Light("Living Room");
        ...
        LightOnCommand livingRoomLightOn =
        new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
        new LightOffCommand(livingRoomLight);
        ...
        remoteControl.setCommand(0, () -> livingRoomLight.on(),
                                () -> livingRoomLight.off());
        ...
    }
}
```

We create the Light object like normal...

But we can remove the concrete LightOnCommand and LightOffCommand objects.

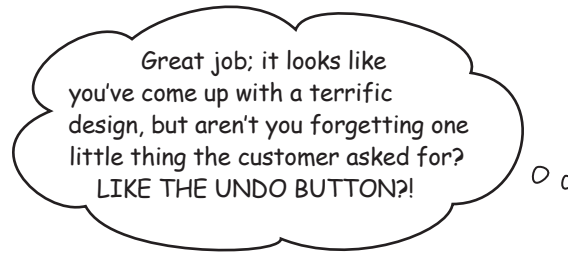
Later, when you click one of the remote's buttons, the remote calls the execute() method of the command object in the slot for that button, which is represented by this lambda expression.

Instead we'll write the concrete commands as lambda expressions that do the same work as the concrete command's execute() method was doing: that is, turning the light on or turning the light off.

Once we've replaced the concrete commands with lambda expressions, we can delete all those concrete command classes (LightOnCommand, LightOffCommand, HottubOnCommand, HottubOffCommand, etc.). If you do this for every concrete command, you'll reduce the total number of classes in the remote control application from 22 to 9.

Note that you can only do this if your Command interface has *one* abstract method. As soon as we add a second abstract method, the lambda shorthand no longer works.

If you like this technique, check out your favorite Java reference for more information on the lambda expression.



Whoops! We almost forgot...luckily, once we have our basic Command classes, undo is easy to add. Let's step through adding undo to our commands and to the remote control...



What are we doing?

Okay, we need to add functionality to support the undo button on the remote. It works like this: say the Living Room Light is off and you press the on button on the remote. Obviously the light turns on. Now if you press the undo button, then the last action will be reversed—in this case, the light will turn off. Before we get into more complex examples, let's get the light working with the undo button:

- 1 When commands support undo, they have an `undo()` method that mirrors the `execute()` method. Whatever `execute()` last did, `undo()` reverses. So, before we can add undo to our commands, we need to add an `undo()` method to the Command interface:

```
public interface Command {
    public void execute();
    public void undo();
}
```

Here's the new `undo()` method.

That was simple enough.

Now, let's dive into the Light commands and implement the `undo()` method.

- 2 Let's start with the `LightOnCommand`: if the `LightOnCommand`'s `execute()` method was called, then the `on()` method was last called. We know that `undo()` needs to do the opposite of this by calling the `off()` method.

```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }

    public void undo() {
        light.off();
    }
}
```

execute() turns the light on, so undo() simply turns the light back off.

Piece of cake! Now for the `LightOffCommand`. Here the `undo()` method just needs to call the `Light`'s `on()` method.

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }

    public void undo() {
        light.on();
    }
}
```

And here, undo() turns the light back on.

Could this be any easier? Okay, we aren't done yet; we need to work a little support into the Remote Control to handle tracking the last button pressed and the undo button press.

- 3 To add support for the undo button, we only have to make a few small changes to the Remote Control class. Here's how we're going to do it: we'll add a new instance variable to track the last command invoked; then, whenever the undo button is pressed, we retrieve that command and invoke its `undo()` method.

```
public class RemoteControlWithUndo {
    Command[] onCommands;
    Command[] offCommands;
    Command undoCommand;

    public RemoteControlWithUndo() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        Command noCommand = new NoCommand();
        for(int i=0;i<7;i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
        undoCommand = noCommand;

    }

    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }

    public void onButtonWasPushed(int slot) {
        onCommands[slot].execute();
        undoCommand = onCommands[slot];
    }

    public void offButtonWasPushed(int slot) {
        offCommands[slot].execute();
        undoCommand = offCommands[slot];
    }

    public void undoButtonWasPushed() {
        undoCommand.undo();
    }

    public String toString() {
        // toString code here...
    }
}
```

This is where we'll stash the last command executed for the undo button.

Just like the other slots, undo starts off with a `noCommand`, so pressing undo before any other button won't do anything at all.

When a button is pressed, we take the command and first execute it; then we save a reference to it in the `undoCommand` instance variable. We do this for both on commands and off commands.

When the undo button is pressed, we invoke the `undo()` method of the command stored in `undoCommand`. This undoes the operation of the last command executed.

Update to add `undoCommands`.

Time to QA that Undo button!

Okay, let's rework the test harness a bit to test the undo button:

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();

        Light livingRoomLight = new Light("Living Room");
        LightOnCommand livingRoomLightOn =
            new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
            new LightOffCommand(livingRoomLight);

        remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);

        remoteControl.onButtonWasPushed(0);
        remoteControl.offButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
        remoteControl.offButtonWasPushed(0);
        remoteControl.onButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
    }
}
```

← Create a Light, and our new undo() enabled Light On and Off Commands.

↻ Add the light Commands to the remote in slot 0.

Turn the light on, then off, and then undo.

Then, turn the light off, back on, and undo.

And here are the test results...

```
File Edit Window Help UndoCommandsDefyEntropy
% java RemoteLoader
Light is on
Light is off

----- Remote Control -----
[slot 0] LightOnCommand      LightOffCommand
[slot 1] NoCommand          NoCommand
[slot 2] NoCommand          NoCommand
[slot 3] NoCommand          NoCommand
[slot 4] NoCommand          NoCommand
[slot 5] NoCommand          NoCommand
[slot 6] NoCommand          NoCommand
[undo] LightOffCommand

Light is on
Light is off
Light is on

----- Remote Control -----
[slot 0] LightOnCommand      LightOffCommand
[slot 1] NoCommand          NoCommand
[slot 2] NoCommand          NoCommand
[slot 3] NoCommand          NoCommand
[slot 4] NoCommand          NoCommand
[slot 5] NoCommand          NoCommand
[slot 6] NoCommand          NoCommand
[undo] LightOnCommand

Light is off
```

Turn the light on, then off.

Here are the Light commands.

Undo was pressed... the LightOffCommand undo() turns the light back on.

Now undo holds the LightOffCommand, the last command invoked.

Then we turn the light off and back on.

Now undo holds the LightOnCommand, the last command invoked.

Undo was pressed, so the light is back off.

Using state to implement Undo

Okay, implementing undo on the Light was instructive but a little too easy. Typically, we need to manage a bit of state to implement undo. Let's try something a little more interesting, like the CeilingFan from the vendor classes. The CeilingFan class allows a number of speeds to be set along with an off method.

Here's the source code for the CeilingFan class:

```
public class CeilingFan {
    public static final int HIGH = 3;
    public static final int MEDIUM = 2;
    public static final int LOW = 1;
    public static final int OFF = 0;
    String location;
    int speed;

    public CeilingFan(String location) {
        this.location = location;
        speed = OFF;
    }

    public void high() {
        speed = HIGH;
        // code to set fan to high
    }

    public void medium() {
        speed = MEDIUM;
        // code to set fan to medium
    }

    public void low() {
        speed = LOW;
        // code to set fan to low
    }

    public void off() {
        speed = OFF;
        // code to turn fan off
    }

    public int getSpeed() {
        return speed;
    }
}
```

CeilingFan
high()
medium()
low()
off()
getSpeed()

Notice that the CeilingFan class holds local state representing the speed of the ceiling fan.

Hmm, so to properly implement undo, I'd have to take the previous speed of the ceiling fan into account...

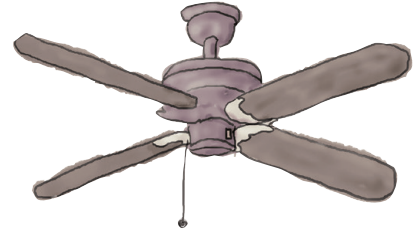
These methods set the speed of the ceiling fan.

We can get the current speed of the ceiling fan using getSpeed().



Adding Undo to the Ceiling Fan commands

Now let's tackle adding undo to the various Ceiling Fan commands. To do so, we need to track the last speed setting of the fan and, if the `undo()` method is called, restore the fan to its previous setting. Here's the code for the `CeilingFanHighCommand`:



```
public class CeilingFanHighCommand implements Command {
    CeilingFan ceilingFan;
    int prevSpeed;

    public CeilingFanHighCommand(CeilingFan ceilingFan) {
        this.ceilingFan = ceilingFan;
    }

    public void execute() {
        prevSpeed = ceilingFan.getSpeed();
        ceilingFan.high();
    }

    public void undo() {
        if (prevSpeed == CeilingFan.HIGH) {
            ceilingFan.high();
        } else if (prevSpeed == CeilingFan.MEDIUM) {
            ceilingFan.medium();
        } else if (prevSpeed == CeilingFan.LOW) {
            ceilingFan.low();
        } else if (prevSpeed == CeilingFan.OFF) {
            ceilingFan.off();
        }
    }
}
```

We've added local state to keep track of the previous speed of the fan.

In `execute()`, before we change the speed of the fan, we need to first record its previous state, just in case we need to undo our actions.

To undo, we set the speed of the fan back to its previous speed.

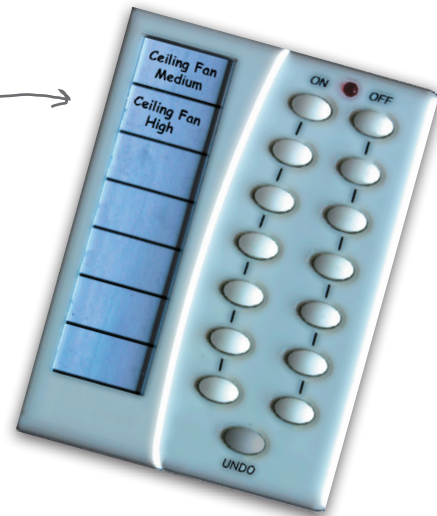


We've got three more ceiling fan commands to write: low, medium, and off. Can you see how these are implemented?

Get ready to test the ceiling fan

Time to load up our remote control with the ceiling fan commands. We're going to load slot 0's on button with the medium setting for the fan and slot 1 with the high setting. Both corresponding off buttons will hold the ceiling fan off command.

Here's our test script:



```
public class RemoteLoader {
```

```
    public static void main(String[] args) {
```

```
        RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();
```

```
        CeilingFan ceilingFan = new CeilingFan("Living Room");
```

```
        CeilingFanMediumCommand ceilingFanMedium =
```

```
            new CeilingFanMediumCommand(ceilingFan);
```

```
        CeilingFanHighCommand ceilingFanHigh =
```

```
            new CeilingFanHighCommand(ceilingFan);
```

```
        CeilingFanOffCommand ceilingFanOff =
```

```
            new CeilingFanOffCommand(ceilingFan);
```

Here we instantiate three commands: medium, high, and off.

```
        remoteControl.setCommand(0, ceilingFanMedium, ceilingFanOff);
```

```
        remoteControl.setCommand(1, ceilingFanHigh, ceilingFanOff);
```

Here we put medium in slot 0, and high in slot 1. We also load up the off command.

```
        remoteControl.onButtonWasPushed(0);
```

First, turn the fan on medium.

```
        remoteControl.offButtonWasPushed(0);
```

Then turn it off.

```
        System.out.println(remoteControl);
```

```
        remoteControl.undoButtonWasPushed();
```

Undo! It should go back to medium...

```
        remoteControl.onButtonWasPushed(1);
```

Turn it on to high this time.

```
        System.out.println(remoteControl);
```

```
        remoteControl.undoButtonWasPushed();
```

And, one more undo; it should go back to medium.

```
    }
```

```
}
```

Testing the ceiling fan...

Okay, let's fire up the remote, load it with commands, and push some buttons!

```
File Edit Window Help UndoThis!

% java RemoteLoader

Living Room ceiling fan is on medium
Living Room ceiling fan is off

----- Remote Control -----
[slot 0] CeilingFanMediumCommand   CeilingFanOffCommand
[slot 1] CeilingFanHighCommand    CeilingFanOffCommand
[slot 2] NoCommand                NoCommand
[slot 3] NoCommand                NoCommand
[slot 4] NoCommand                NoCommand
[slot 5] NoCommand                NoCommand
[slot 6] NoCommand                NoCommand
[undo] CeilingFanOffCommand

Living Room ceiling fan is on medium
Living Room ceiling fan is on high

----- Remote Control -----
[slot 0] CeilingFanMediumCommand   CeilingFanOffCommand
[slot 1] CeilingFanHighCommand    CeilingFanOffCommand
[slot 2] NoCommand                NoCommand
[slot 3] NoCommand                NoCommand
[slot 4] NoCommand                NoCommand
[slot 5] NoCommand                NoCommand
[slot 6] NoCommand                NoCommand
[undo] CeilingFanHighCommand

Living Room ceiling fan is on medium

%
```

Turn the ceiling fan on medium, then turn it off.

Here are the commands in the remote control...

...and undo has the last command executed, the CeilingFanOffCommand, with the previous speed of medium.

Undo the last command, and it goes back to medium.

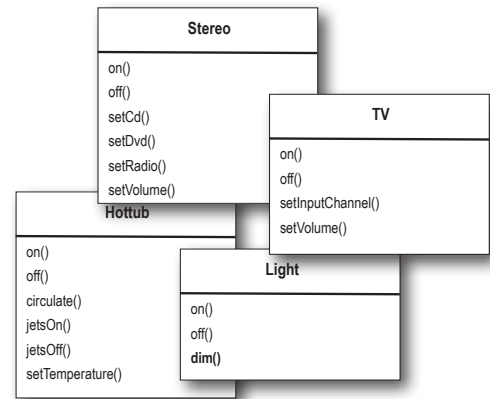
Now, turn it on high.

Now, high is the last command executed.

One more undo, and the ceiling fan goes back to medium speed.

Every remote needs a Party Mode!

What's the point of having a remote if you can't push one button and have the lights dimmed, the stereo and TV turned on, and the hot tub fired up?



Hmm, our remote control would need a button for each device, so I don't think we can do this.

Hold on, Sue, don't be so sure. I think we can do this without changing the remote at all!

Mary's idea is to make a new kind of Command that can execute other Commands... and more than one of them! Pretty good idea, huh?

```
public class MacroCommand implements Command {
    Command[] commands;
```

```
    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }
```

Take an array of Commands and store them in the MacroCommand.

```
    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }
}
```

When the macro gets executed by the remote, execute those commands one at a time.

Using a macro command

Let's step through how we use a macro command:

- 1 First we create the set of commands we want to go into the macro:

```
Light light = new Light("Living Room");
TV tv = new TV("Living Room");
Stereo stereo = new Stereo("Living Room");
Hottub hottub = new Hottub();

LightOnCommand lightOn = new LightOnCommand(light);
StereoOnCommand stereoOn = new StereoOnCommand(stereo);
TVOnCommand tvOn = new TVOnCommand(tv);
HottubOnCommand hottubOn = new HottubOnCommand(hottub);
```

Create all the devices: a light, tv, stereo, and hot tub.

Now create all the On commands to control them.



Sharpen your pencil

We'll also need commands for the off buttons.
Write the code to create those here:

- 2 Next we create two arrays, one for the On commands and one for the Off commands, and load them with the corresponding commands:

```
Command[] partyOn = { lightOn, stereoOn, tvOn, hottubOn};
Command[] partyOff = { lightOff, stereoOff, tvOff, hottubOff};

MacroCommand partyOnMacro = new MacroCommand(partyOn);
MacroCommand partyOffMacro = new MacroCommand(partyOff);
```

Create an array for On commands and an array for Off commands...

...and create two corresponding macros to hold them.

- 3 Then we assign MacroCommand to a button like we always do:

```
remoteControl.setCommand(0, partyOnMacro, partyOffMacro);
```

Assign the macro command to a button as you would any command.

- 4 Finally, we just need to push some buttons and see if this works.

```
System.out.println(remoteControl);
System.out.println("--- Pushing Macro On---");
remoteControl.onButtonWasPushed(0);
System.out.println("--- Pushing Macro Off---");
remoteControl.offButtonWasPushed(0);
```

Here's the output.

```
File Edit Window Help You Can't Beat ABabka
% java RemoteLoader
----- Remote Control -----
[slot 0] MacroCommand      MacroCommand
[slot 1] NoCommand         NoCommand
[slot 2] NoCommand         NoCommand
[slot 3] NoCommand         NoCommand
[slot 4] NoCommand         NoCommand
[slot 5] NoCommand         NoCommand
[slot 6] NoCommand         NoCommand
[undo] NoCommand

--- Pushing Macro On---
Light is on
Living Room stereo is on
Living Room TV is on
Living Room TV channel is set for DVD
Hottub is heating to a steaming 104 degrees
Hottub is bubbling!

--- Pushing Macro Off---
Light is off
Living Room stereo is off
Living Room TV is off
Hottub is cooling to 98 degrees
```

Here are the two macro commands.

All the Commands in the macro are executed when we invoke the on macro...

...and when we invoke the off macro. Looks like it works.



Exercise

The only thing our MacroCommand is missing is its undo functionality. When the undo button is pressed after a macro command, all the commands that were invoked in the macro must undo their previous actions. Here's the code for MacroCommand; go ahead and implement the undo() method:

```
public class MacroCommand implements Command {
    Command[] commands;

    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }

    public void undo() {
        
    }
}
```

there are no Dumb Questions

Q: Do I always need a receiver? Why can't the command object implement the details of the execute() method?

A: In general, we strive for "dumb" command objects that just invoke an action on a receiver; however, there are many examples of "smart" command objects that implement most, if not all, of the logic needed to carry out a request. Certainly you can do this; just keep in mind you'll no longer have the same level of decoupling between the invoker and receiver, nor will you be able to parameterize your commands with receivers.

Q: How can I implement a history of undo operations? In other words, I want to be able to press the undo button multiple times.

A: Great question. It's pretty easy actually; instead of keeping just a reference to the last Command executed, you keep a stack of previous commands. Then, whenever undo is pressed, your invoker pops the first item off the stack and calls its undo() method.

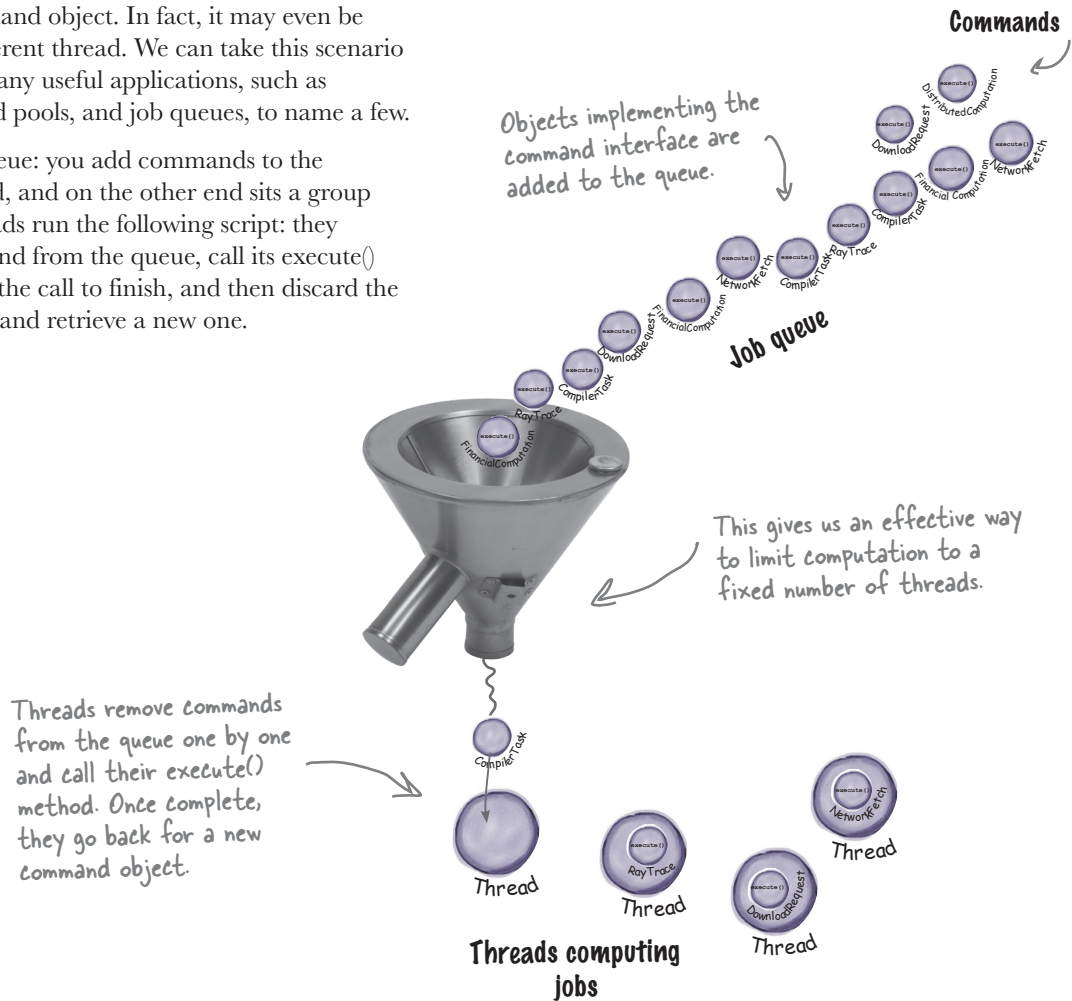
Q: Could I have just implemented party mode as a Command by creating a PartyCommand and putting the calls to execute the other Commands in PartyCommand's execute() method?

A: You could; however, you'd essentially be "hardcoding" the party mode into PartyCommand. Why go to the trouble? With MacroCommand, you can decide dynamically which Commands you want to go into PartyCommand, so you have more flexibility using MacroCommands. In general, MacroCommand is a more elegant solution and requires less new code.

More uses of the Command Pattern: queuing requests

Commands give us a way to package a piece of computation (a receiver and a set of actions) and pass it around as a first-class object. Now, the computation itself may be invoked long after some client application creates the command object. In fact, it may even be invoked by a different thread. We can take this scenario and apply it to many useful applications, such as schedulers, thread pools, and job queues, to name a few.

Imagine a job queue: you add commands to the queue on one end, and on the other end sits a group of threads. Threads run the following script: they remove a command from the queue, call its `execute()` method, wait for the call to finish, and then discard the command object and retrieve a new one.



Note that the job queue classes are totally decoupled from the objects that are doing the computation. One minute a thread may be computing a financial computation, and the next it may be retrieving something from the network. The job queue objects don't care; they just retrieve commands and call `execute()`. Likewise, as long as you put objects into the queue that implement the Command Pattern, your `execute()` method will be invoked when a thread is available.



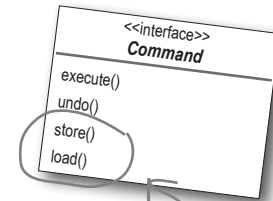
How might a web server make use of such a queue? What other applications can you think of?

More uses of the Command Pattern: logging requests

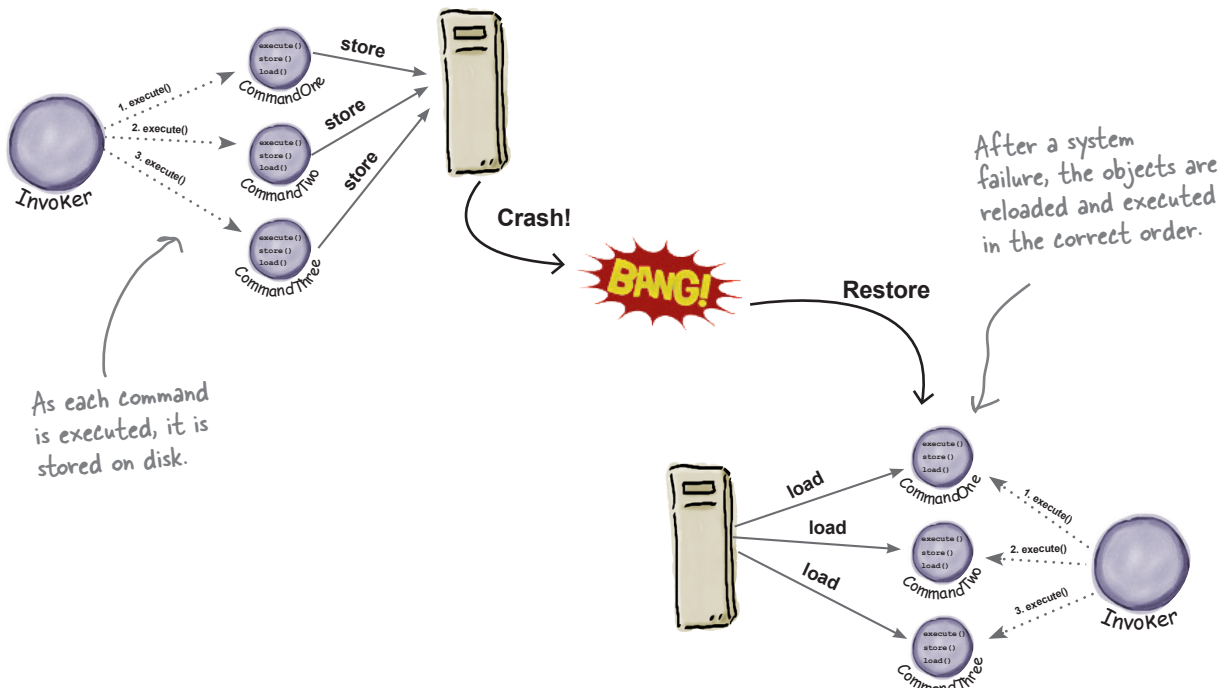
The semantics of some applications require that we log all actions and be able to recover after a crash by reinvoking those actions. The Command Pattern can support these semantics with the addition of two methods: `store()` and `load()`. In Java we could use object serialization to implement these methods, but the normal caveats for using serialization for persistence apply.

How does this work? As we execute commands, we store a history of them on disk. When a crash occurs, we reload the command objects and invoke their `execute()` methods in batch and in order.

Now, this kind of logging wouldn't make sense for a remote control; however, there are many applications that invoke actions on large data structures that can't be quickly saved each time a change is made. By using logging, we can save all the operations since the last checkpoint, and if there is a system failure, apply those operations to our checkpoint. Take, for example, a spreadsheet application: we might want to implement our failure recovery by logging the actions on the spreadsheet rather than writing a copy of the spreadsheet to disk every time a change occurs. In more advanced applications, these techniques can be extended to apply to sets of operations in a transactional manner so that all of the operations complete, or none of them do.



We add two methods for logging.



Command Pattern in the Real World

Remember the little life-changing application from Chapter 2?

In that chapter we saw how Java's Swing library is chock full of Observers in the form of ActionListeners that listen in (or *observe*) events on user interface components.

Well, it turns out that ActionListener is not just an Observer interface, it's also a Command interface, and our AngelListener and DevilListener classes are not just Observers, but also concrete Commands. That's right, we have two patterns in one example!



Sharpen your pencil

Here's the code (the important bits anyway) for the little life-changing application from Chapter 2. See if you can identify who is the Client, who are the Commands, who is the Invoker, and who is the Receiver.

```
public class SwingObserverExample {
    // Set up ...
    JButton button = new JButton("Should I do it?");
    button.addActionListener(new AngelListener());
    button.addActionListener(new DevilListener());
    // Set frame properties here
}

class AngelListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Don't do it, you might regret it!");
    }
}

class DevilListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Come on, do it!");
    }
}
}
```

Sharpen your pencil Solution

Here's the code (the important bits anyway) for the little life-changing application from Chapter 2. See if you can identify who is the Client, who are the Commands, who is the Invoker, and who is the Receiver?

Here's our solution.

```
public class SwingObserverExample {
    // Set up ...
```

```
    JButton button = new JButton("Should I do it?");
    button.addActionListener(new AngelListener());
    button.addActionListener(new DevilListener());
```

```
    // Set frame properties here
}
```

```
class AngelListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Don't do it, you might regret it!");
    }
}
```

```
class DevilListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Come on, do it!");
    }
}
```

The Receiver in this example is the System object. Remember, invoking a command results in actions on the Receiver. In a typical Swing application this would result in calling actions on other components in the UI.

The button is our Invoker. The button calls the actionPerformed() (like execute()) methods in the commands (the ActionListeners) when you click the button.

The Client is the class that sets up the Swing components and sets the commands (AngelListener and DevilListener) in the Invoker (the Button).

ActionListener is the Command Interface: it has one method, actionPerformed() that, like execute(), is executed when the command is invoked.

AngelListener and DevilListener are our concrete Commands. They implement the command interface (in this case, ActionListener).